

HaaS Backend Testing

David Cho

Electrical and Computer Engineering
Austin, TX 78712
dyc325@eid.utexas.edu

Eduardo Rosales

Electrical and Computer Engineering
Austin, TX 78712
er36268@eid.utexas.edu

Anita Woodford

Electrical and Computer Engineering
Austin, TX 78712
ajw4987@eid.utexas.edu

Abstract— This study evaluated the extent to which formal testing criteria improved the effectiveness of a constrained baseline test suite for a cloud-native backend, with emphasis on user authentication and hardware check-out/check-in workflows. The baseline suite was deliberately limited to a small set of behavior-oriented tests that were derived from endpoint specifications, after which prime-path structural testing, Correlated Active Clause Coverage (CACC), input space partitioning, and mutation analysis were applied across three targets. The results indicated that formal criteria contributed more substantially to defect identification than to aggregate improvements in coverage or mutation scores. Specifically, structural and partition-based testing revealed defects not exposed by the baseline suite, including an infeasible branch, acceptance of invalid inputs, and an unhandled crash path, while prime-path testing exercised error-handling branches not reached under baseline testing. Quantitative improvements in coverage and mutation performance were modest and dependent on the scope of measurement; notably, a 100% mutation kill rate for `_encrypt()` contrasted with only a limited baseline-to-partition difference for the authentication endpoints, a result influenced in part by the test infrastructure design rather than technique equivalence. Overall, the findings suggest that formal testing criteria provide meaningful gains in defect detection and in identification of inadequately tested logic, even when corresponding improvements in summary metrics remain limited.

Keywords— *Base Choice Coverage, Correlated Active Clause Coverage, input space partitioning, mutation analysis, prime path coverage, software testing*

I. INTRODUCTION

Software testing in modern cloud-native systems must address both functional correctness and the complex control logic that arises from input validation, branching behavior, and multi-step service workflows. In practice, many development teams rely primarily on compact behavior-driven test suites that confirm expected outputs for common scenarios, but such suites do not necessarily test structurally significant paths or reveal subtle logic defects. As a result, important faults may remain undetected even when baseline tests appear adequate by conventional pass/fail standards.

Formal testing criteria provide a more disciplined basis for evaluating test adequacy. Structural techniques such as prime-path coverage target control-flow behavior directly, while logic-based criteria such as Correlated Active Clause Coverage (CACC) examine the behavior of compound predicates. Input space partitioning complements these

approaches by organizing the input domain into meaningful classes and selecting representative test cases systematically. Mutation analysis further strengthens evaluation by measuring whether a test suite can distinguish the implemented program from fault-injected variants. Together, these methods offer a way to assess not only whether software passes tests, but also how thoroughly its logic and input handling have been examined.

This paper investigated the extent to which these formal criteria improved testing effectiveness relative to a constrained baseline test suite in a cloud-native backend context. Focusing on authentication and hardware checkout/checkin workflows, the study compared baseline and criteria-based testing in terms of defect detection, test coverage, and mutation score. The goal was to determine whether formal methods produce measurable improvements in coverage and defect detection.

II. PROJECT DESCRIPTION AND TEST SUITE IMPLEMENTATION

This project examines the testing of a cloud-native backend that supports user authentication and hardware checkout/checkin workflows. The objective was not only to validate expected behavior, but also to compare a constrained baseline test suite against several formal testing approaches in order to assess their relative effectiveness. To support that comparison, the project was organized around three representative targets: the `_encrypt()` helper in the authentication module, the `register()` and `login()` authentication endpoints, and the hardware checkout/checkin logic. These targets were selected because they contain a mix of input validation, branching behavior, and state-dependent outcomes that make them suitable for structural, logic-based, and input-focused analyses.

The test suite implementation was developed in phases so that comparisons would remain traceable. A small baseline suite for each target was created first using a standardized developer-style approach. Baseline tests were derived from specification intent and directly observable behavior, or from reading the code, rather than from formal structural artifacts such as control flow graphs (CFGs), prime paths, or clause tables. To keep the baseline suites consistent across all targets, the following rules were applied:

- Baseline design is limited to a small, fixed effort budget, targeting 4 tests per function or endpoint
- Each target follows a common scenario template: one happy-path case, one common invalid-input case, one common missing-field or unauthorized case when applicable, and one boundary, guard, or behavioral case.

- Tests are not derived from CFGs, path enumeration, or logic-coverage tables
- Baseline and criteria-based suites must use the same fixtures, database seeding strategy, and mocking approach (In practice, each target maintained its own `confest.py`; their implementations diverge in `$pull` handling for array-of-objects fields, but do not affect the auth endpoint tests or mutation analyses in the report)
- Baseline tests are required to include meaningful assertions over status codes, responses, or persisted state

After the baseline suites were established and coverage was recorded, separate suites were implemented for structural coverage, CACC-based logic analysis, input space partitioning, and mutation analysis. This separation made it possible to evaluate how each method contributed to coverage and defect detection. The following describes the implementation and rationale for each formal-criteria suite:

A. Structural testing

Prime-path coverage was applied to `_encrypt()` and the `checkout/checkin` endpoints. A control flow graph was first constructed for each target by mapping the implementation branches, guards, and return points to nodes and edges. All prime paths, maximal simple paths through the CFG, were enumerated, and test requirements were derived from paths not already satisfied by the baseline. Test cases were then written to satisfy the remaining requirements, targeting error-handling branches and post-update fetch failures the baseline had not reached.

B. Logic coverage (CACC)

CACC was applied to the key predicates governing availability in `checkout_hardware()` and the amount guard in `checkin_hardware()`, as well as the validation predicates in `_encrypt()`. For each predicate, the major and minor clauses were identified, and test pairs were constructed such that the major clause independently determines the predicate outcome (true in one test, false in the other) while all minor clauses are held at values that satisfy the determination condition.

C. Input space partitioning (ISP)

ISP was applied to all three targets using Base Choice Coverage (BCC). For each target, input characteristics were identified from endpoint specifications and function signatures and partitioned into complete and disjoint blocks. A base block was then selected for each characteristic, and the cross-product of base choices formed the base test frame. One additional test frame was then generated per non-base block, varying that characteristic while holding all others at their base choice. Test cases were then implemented from these frames and executed against the implementation; observed behavior was compared against expected behavior derived from the specification to identify defects.

D. Mutation testing

Mutation testing was conducted using `mutmut`, which systematically introduced single-operator mutations into the source code of the targeted modules. Two separate runs were performed: one against the baseline suite and one against the ISP suite for the `login()/register()` target. One additional run was performed for `_encrypt()`. After each run, surviving mutants were inspected and analyzed using the RIP model: for each divergent case, reachability, infectiousness, and propagation were assessed to determine whether the survival indicated a genuine test gap or a structural constraint such as an infeasible infection path.

All suites share a common test infrastructure: a `FakeCollection/FakeDB` in-memory test double replacing `MongoDB`, patched at the module level via `unittest.mock.patch`. The baseline suite applied a four-test-per-target template (happy path, invalid input, missing or unauthorized, boundary or stateful) without reference to implementation structure. Below is Table 1 that summarizes the test suites that were implemented against each target:

Table 1 TEST SUITE SUMMARY

Target	Suite	Criterion	Tests
<code>_encrypt()</code>	Baseline	Ad hoc template	4
<code>login(), register()</code>	Baseline	Ad hoc template	8
<code>checkout_hardware(), checkin_hardware()</code>	Baseline	Ad hoc template	8
<code>_encrypt()</code>	ISP	Base Choice Coverage	7
<code>login(), register()</code>	ISP	Base Choice Coverage	17
<code>checkout_hardware(), checkin_hardware()</code>	ISP	Base Choice Coverage	8
<code>_encrypt()</code>	Structural	Prime Path Coverage	9
<code>checkout_hardware(), checkin_hardware()</code>	Structural	Prime Path Coverage	12
<code>_encrypt()</code>	CACC	Correlated Active Clause	4
<code>checkout_hardware(), checkin_hardware()</code>	CACC	Correlated Active Clause	7

III. RESULTS AND ANALYSES

This section presents the results of applying structural testing, input space partitioning, CACC-based logic coverage, and mutation analysis to the three selected targets. Results are organized across three sections: defects identified through formal criteria that were not exposed by the baseline, branch coverage achieved by each suite per target, and mutation scores comparing the baseline and ISP suites for the authentication endpoints. Together, these results illustrate that formal criteria contributed meaningfully to defect detection even when producing modest improvements in aggregate coverage and mutation metrics.

A. Defect detection

Input space partitioning contributed most directly to defect discovery across the authentication targets. By systematically

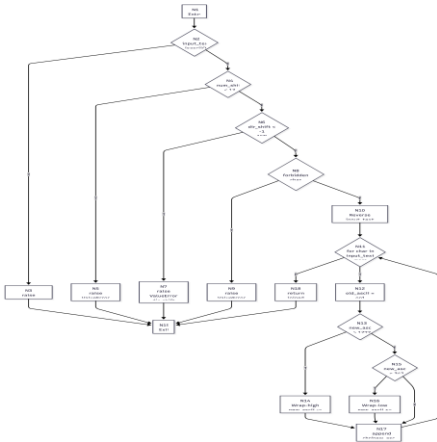
deriving non-base test frames from explicitly defined input characteristics and partition blocks, the ISP suite exercised semantically invalid but structurally present inputs, such as empty strings and null values, that were not covered by the baseline suite. All seven defects identified across `register()`, `login()`, and `_encrypt()` were exposed through execution of these non-base test frames, where the observed behavior diverged from the intended specification. These defects were not revealed by the baseline approach because its invalid-input cases focused primarily on missing fields rather than invalid values supplied in otherwise present fields. The most severe defect was an unhandled server crash in `register()` caused by a null password, a case that required an explicitly modeled non-base input block (C2: b4) to reach and observe.

B. Defect Traceability Across Three Methods

Defect D-7 (invalid `dir_shift` values accepted by `_encrypt()`) was detected and confirmed through three independent formal methods, providing strong evidence of its validity and root cause. First, input space partitioning under Base Choice Coverage with non-base partition block C3: b3 directly exposed the defect by exercising `dir_shift = 2`, an int that should trigger a `ValueError` but was silently accepted. Second, structural analysis of the control flow graph revealed that the validation predicate `dir_shift < -1 AND dir_shift > 1` at line 26 is unsatisfiable; no int value can simultaneously be less than -1 and greater than 1, making the true branch unreachable and leaving Node N7 and Edge E6 is infeasibility by construction. This infeasibility represents a logic defect, not merely a coverage gap, because it indicates that the validation guard fails to properly reject invalid inputs over a range of values. Third, CACC-based clause determination testing confirmed the infeasibility analytically: this requirement “`c1=T` and `c2=F`” (where `c1 = dir_shift < -1` and `c2 = dir_shift > 1`) is structurally impossible, reinforcing the partition-based finding and demonstrating that the defect arises from an unsatisfiable predicate condition rather than from inadequate test coverage. These three methods provide complementary evidence that D-7 is a genuine control-logic defect and not an artifact of test design or tool limitation.

Table 2 Compact CFG Feasibility Metric

Metric	<code>_encrypt()</code>
Total Nodes	19
Feasible Nodes	18
Infeasible Nodes	1 (N7)
Total Edges	21
Feasible Edges	20
Infeasible Edges	1 (E6)
Prime Paths Identified	8
Feasible Prime Paths	7
Structural Coverage Achieved	100% feasible node, edge, and prime-path coverage



Input space partition using the Base Choice Coverage (BCC) strategy further contributed to defect detection by exercising boundary and invalid partitions that the baseline omitted. The input domain model defined three characteristics, amount value (five blocks: negative, zero, minimum valid, exact capacity, and over capacity), user assignment (assigned vs. not assigned), and project existence (valid vs. invalid), yielding eight BCC tests per function. Notably, the exact-capacity block (C1 = b4) confirmed that checking out all the available units succeeds and leaves availability at zero, while the over-capacity block (C1 = b5) confirmed that the availability guard correctly returns a 400 status. These partition-driven tests complement the structural and CACC suites by probing input boundaries that are otherwise easy to overlook.

C. Test coverage

Coverage was measured using pytest with --cov-branch to compare the constrained baseline suites with the criteria-based suites on the selected backend targets. For the authentication module, baseline testing of register() and login() produced strong initial coverage of the primary success and error-handling behaviors, contributing to an overall 84% statement coverage result for auth.py, and 88% when combined with the baseline _encrypt() tests. The partition-based suite achieved 81% coverage for the full auth.py module. Although this did not represent a large numerical increase, the partition tests expanded coverage in a more meaningful way by exercising semantically invalid but structurally present inputs, such as empty strings and null values for userId and password, that were not represented in the baseline suite. As a result, the formal suite provided deeper coverage of validation behavior in register() and login(), even where the overall module-level percentage changed only modestly.

Branch coverage for the hardware endpoints was measured against backend/app/routes/hardware.py (153 statements, 44 branches). The baseline suite (8 tests) achieved 49% coverage, the CACC suite (7 tests) 48%, the ISP/BCC suite (8 tests) 48%, and the structural suite (12 tests) 52%. The structural suite attained the highest coverage by directly targeting previously uncovered prime paths, including invalid ObjectId handling, not-found responses, the existing-entry increment path, and the post-update fetch failure.

Partially covered branches dropped from 10 in the baseline to 4 under the structural suite, confirming its efficiency at closing branch gaps. Remaining uncovered lines correspond to other route handlers in the hardware.py (list, get, update, delete) that fall outside the scope of this analysis. Overall, the structural suite's targeted path selection produced the most meaningful gains in branch completeness.

Table 3 Master Metrics Summary Across All Targets and Suites

Target	Suite	Criterion	Tests	Coverage Scope	Mutation Scope	Defects Found
_encrypt()	Baseline	Ad hoc template	4	auth.py baseline contribution; baseline-only merge behavior	None	0
_encrypt()	ISP	Base Choice Coverage	7	auth.py criteria phase; partitions for edge, not, none_shift, dir_shift	None	1 (D-7)
_encrypt()	Structural	Prime Path Coverage	9	100% branch node, edge, and prime-path coverage; 17 paths, 21 edges (N, Nk infanodes)	None	1 (D-7 support via infeasibility)
_encrypt()	CACC	Correlated Active Choice Coverage	4	Predicate-level infeasibility analysis for dir_shift; supports interpretation of branch coverage	None	1 (D-7 support via unsatisfiable predicate)
_encrypt()	Mutation RFP	Manual mutant analysis	4 tests, 4 mutants	Encrypt-focused mutation; RFP analysis	0 mutants, 4 killed, 100% diff rate	0 new defects; confirms logic sensitivity
login(), register()	Baseline	Ad hoc template	8	auth.py baseline endpoint coverage	Manual	0
login(), register()	ISP	Base Choice Coverage	17	auth.py partition-based endpoint coverage	Manual RFP	0 (D-1 to D-6)
checkout_hardware(), checkin_hardware()	Baseline	Ad hoc template	8	hardware.py baseline branch coverage	None	0
checkout_hardware(), checkin_hardware()	ISP	Base Choice Coverage	8	hardware.py partition-based branch coverage	None	0 new defects
checkout_hardware(), checkin_hardware()	Structural	Prime Path Coverage	12	hardware.py targeted prime-path coverage expansion (10 paths for checkout, 1 for checkin)	None	0 new functional defects; structural gaps exposed
checkout_hardware(), checkin_hardware()	CACC	Correlated Active Choice Coverage	7	Availability predicate logic and infeasibility context	None	0 new functional defects; infeasibility obligations clarified

Total test count: 88 tests across all targets and suites (baseline 28 + structural 21 + CACC 11 + ISP 32 + mutation 4 encrypt-focused)

Mutation score

Mutation testing was used to evaluate fault-detection effectiveness across the project's selected authentication targets by comparing baseline and criteria-based suites under injected faults. For the register() and login() endpoints, the aggregate difference between the baseline and partition-based suites was small: the baseline suite killed 77 mutants and left 44 surviving (63.6%), while the partition-based suite killed 75 and left 46 surviving (62.0%). However, the mutant-level analysis showed that this narrow gap was partly due to fixture behavior and input-model scope rather than equivalent test effectiveness. For example, the mutant replacing bp = Blueprint("auth", __name__) with bp = None was killed by the partition suite but survived the baseline suite because the baseline's broader mocking setup absorbed the initialization failure before it propagated to an observable assertion. Conversely, a mutant changing the login lookup from db["users"] to db["XXusersXX"] was killed by the baseline suite but survived the partition suite because the partition tests used a MagicMock database object that returned the same collection for any key, preventing infection from propagating. The mutation results also exposed an input-modeling gap in the partition suite: two register() mutants affecting the empty-body response survived because the partition model did not include {} as a distinct input class, whereas the baseline suite did.

The _encrypt() mutation analysis showed stronger sensitivity to localized logic and boundary faults. Four manually designed mutants targeted the num_shift guard, the unsatisfiable dir_shift predicate, and the upper and lower wrap-around boundaries. These mutants were reported as killed, indicating that the _encrypt() criteria-based tests were effective at detecting boundary and logic faults in a tightly scoped unit. In particular, the mutation changing dir_shift < -1 and dir_shift > 1 to dir_shift < -1 or dir_shift > 1 was killed because tests explicitly exercised invalid dir_shift values and exposed the original validation defect. The wrap-around mutations were also killed by tests reaching the exact

boundary values 127 and 34, demonstrating sensitivity to off-by-one errors in the cipher logic. Overall, mutation analysis showed that formal criteria were more effective for compact logic-heavy code such as `_encrypt()` than for endpoint behavior whose mutation score was influenced by fixture design and incomplete input modeling. This makes mutation testing useful not only as a comparative score, but also as a diagnostic tool for identifying weaknesses in test infrastructure, input-domain assumptions, and fault-propagation conditions.

Table 4: `_encrypt()` Mutation Mini-Table(RIP Outcome)

Mutant ID	Mutation	Expected Fault Type	Execution Outcome	RIP Assessment	Kill Status
M1	Changed <code>num_shift < 1</code> to <code>num_shift == 1</code>	Boundary condition violation (except 1, should reject 1)	16/61 runs failed; encrypted output differs for boundary input	Reachable (R/F), Infeasible (I/F), Propagating (P/F)	Killed
M2	Changed <code>dir_shift < 1</code> AND <code>dir_shift > 1</code> to <code>dir_shift < 1 OR dir_shift > 1</code>	Logic condition overruns (faulty predicate: <code>num_shift</code>)	4/39 runs failed; invalid <code>dir_shift</code> now rejected	Reachable (R/F), Infeasible (I/F), Propagating (P/F)	Killed
M3	Changed upper boundary <code>>= 127</code> to <code>>= 128</code> at <code>shift127</code> wrap point	Off-by-one at wrap-high boundary	2/39 runs failed; <code>shift</code> overruns invalid argument	Reachable (R/F), Infeasible (I/F), Propagating (P/F)	Killed
M4	Changed lower boundary <code><= 34</code> to <code><= 35</code> at <code>shift34</code> wrap point	Off-by-one at wrap-low boundary	1/39 runs failed; boundary wrap assertion fails	Reachable (R/F), Infeasible (I/F), Propagating (P/F)	Killed

Mutation score for `_encrypt`: 4/4 killed 100%.

Table 5: Defects Found

ID	Target	Input / Condition	Intended	Observed	Suite
D-1	register()	userid: ""	400	201: empty string stored as userid	ISP (C1:b3)
D-2	register()	userid: null	400	201: null stored as userid	ISP (C1:b4)
D-3	register()	password: ""	400	201: empty password stored	ISP (C2:b3)
D-4	register()	password: null	400	500: unhandled crash (AttributeError)	ISP (C2:b4)
D-5	login()	userid: ""	400	401: misclassified as invalid credentials	ISP (C1:b3)
D-6	login()	password: ""	400	401: misclassified as invalid credentials	ISP (C2:b3)
D-7	_encrypt()	dir_shift: 2	ValueError	No error raised	ISP (C3:b3); Structural (infeasible branch); CACC (unsatisfiable predicate)

IV. FUTURE WORK

Several limitations of the present study suggest clear directions for future work. First, the analysis was restricted to a small number of backend targets, and some results, particularly the mutation results for `register()` and `login()`, were influenced by fixture behavior and incomplete input modeling. Future work should expand the study to additional endpoints and refine the test infrastructure so that mocked database behavior more closely reflects production semantics. This would reduce differences caused by test setup and make

comparisons between baseline and criteria-based suites more reliable.

Second, the project relied primarily on manually constructed criteria-based tests. A natural extension would be to incorporate more widely used contemporary testing tools to improve automation, scale, and realism. For example, Playwright could be used to extend the current backend-focused work into end-to-end testing of realistic user workflows across the application interface. Similarly, Schemathesis could support automated API-level test generation from endpoint specifications, helping address the input-modeling gaps observed in this study by exploring malformed, boundary, and omitted-input cases that were impractical to enumerate manually. These tools would complement the formal techniques used here by making the resulting test process more scalable and closer to current testing practice.

V. CONCLUSION

This study evaluated the extent to which formal testing criteria improve the effectiveness of a constrained baseline test suite applied to a cloud-native backend. Across three representative targets, `_encrypt()`, the authentication endpoints `register()`, and `login()`, and the hardware checkout/checkin workflows, the results indicated that formal criteria contributed more to defect identification than to aggregate improvements in coverage or mutation scores.

Input space partitioning made the strongest direct contribution to defect detection, exposing seven defects across the authentication targets that were not revealed by the baseline suite. All seven were exposed by non-base input blocks, including empty strings, null values, and a case associated with an unsatisfiable validation predicate, that were not modeled as distinct obligations under the baseline scenario template. Structural testing extended coverage of the hardware endpoints by targeting prime paths not exercised under baseline conditions, including error-handling branches that required controlled mock-based fault injection to reach. CACC independently confirmed the `_encrypt()` predicate defect through formal clause analysis, demonstrating that structural infeasibility can be established analytically rather than only observed empirically.

The mutation results for `login()` and `register()` also illustrated a limitation of aggregate metrics. The baseline and ISP suites produced nearly identical scores despite killing different mutants for different reasons, with the small difference explained by infrastructure design and an omitted input block rather than by equivalent test logic. By contrast, the `_encrypt()` mutation analysis achieved a 100% kill rate across all manually designed mutants, suggesting that formal criteria are especially effective for compact, logic-heavy functions whose boundaries and predicates can be modeled precisely.

Taken together, these findings support the conclusion that formal testing criteria provide meaningful improvements in defect detection and in the identification of inadequately tested logic paths, even when their effect on summary metrics

Commented [WA1]: @Cho, David Y I will wait for you to get the outline and stuff figured out i dont want to add to stuff that you are doing, I put some notes down but i will wait. Are you okay with that?

Commented [CD2R1]: Hey@Anita, was working on the abstract and intro..I think if you want you can put in things like your implementation details, results and analyses in the meantime

remains modest. Coverage percentages and mutation scores are useful indicators, but they are not sufficient on their own. Interpreting why a mutant survived or why a branch remained uncovered requires the kind of structural and logical analysis that formal criteria make systematic.

VI. THREATS TO VALIDITY

A. Internal Validity.

Observer differences between baseline and criteria-based suites may be influenced by test infrastructure behavior, especially with mocking and fixture design. For login and register mutation runs, some mutant outcomes were affected by MagicMock collection behavior and patch scope rather than only by test logic strength. To mitigate this, all suites used the same in-memory FakeCollection or FakeDB pattern per target and were executed under the same coverage and mutation tooling workflow.

B. Construct Validity.

Coverage percentage and mutation score were treated as indicators of adequacy, but they do not fully represent the fault detection quality on their own. This study paired metric reporting with structural analysis(CFG and prime paths), logic analysis (CACC), and RIP-based interpretation of mutant behavior. Infeasible obligations, such as the unsatisfiable dir shift branch in _encrypt, were documented explicitly to avoid misclassifying infeasibility as a test gap.

C. External Validity

The study focused on three backend targets in one cloud-native project, so generalization to other systems may be limited. Endpoint behavior and mutation outcomes may vary across frameworks, database layers, and validation libraries. Results are most directly applicable to services with similar input validation structure, branching logic, and mock-driven test infrastructure.

REFERENCES

[1] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2016.

[2] A. Hovmöller, “mutmut,” software, version 2.5.1, Aug. 2024. [Online]. Available: <https://pypi.org/project/mutmut/>. [Accessed: Apr. 16, 2026].

[3] M. Schlaich and I. C. Mărieș, “pytest-cov,” software, version 7.1.0, Mar. 2026. [Online]. Available: <https://pypi.org/project/pytest-cov/>. [Accessed: Apr. 16, 2026].

Commented [WA3]: @Cho, David Y I will wait for you to get the outline and stuff figured out i dont want to add to stuff that you are doing, I put some notes down but i will wait. Are you okay with that?

Commented [CD4R3]: Hey@Anita, was working on the abstract and intro..I think if you want you can put in things like your implementation details, results and analyses in the meantime